

Unit – IV

Processor

Instruction Execution – Building a Data Path – Designing a Control Unit – Pipelining – Data Hazard – Control Hazards.

LOAD R5,X(R7)

- Fetch the instruction from the memory.
- Increment the program counter.
- Decode the instruction to determine the operation to be performed.
- Read register R7.
- Add the immediate value X to the contents of R7.
- Use the sum $X + [R7]$ as the effective address of the source operand, and read the contents of that location in the memory.
- Load the data received from the memory into the destination register, R5.

fetch, decode, operand fetch, Arithmetic and logical unit, and memory store.

1. Fetch the instruction and increment the program counter.
2. Decode the instruction and read the contents of register R7 in the register file.
3. Compute the effective address.
4. Read the memory source operand.
5. Load the operand into the destination register, R5.

ADD R3, R4, R5

ADD R3, R4, #1000

Steps for Add



1. Fetch the instruction and increment the program counter.
2. Decode the instruction and read registers R4 and R5.
3. Compute the sum $[R4] + [R5]$.
4. No action.
5. Load the result into the destination register, R3.

Eg 2. Add R3, R4, #1000

1. Fetch the instruction and increment the program counter.
2. Decode the instruction and read register R4.
3. Compute the sum $[R4] + 1000$.
4. No action.
5. Load the result into the destination register, R3.



STORE R6, X(R8)



Steps for Store

1. Fetch the instruction and increment the program counter.
2. Decode the instruction and read registers R6 and R8.
3. Compute the effective address $X + [R8]$.
4. Store the contents of register R6 into memory location $X + [R8]$.
5. No action.

INSTRUCTION EXECUTION

- Complete instruction cycle – three operations
 - Instruction Fetching
 - Opcode Decoding
 - Instruction Execution

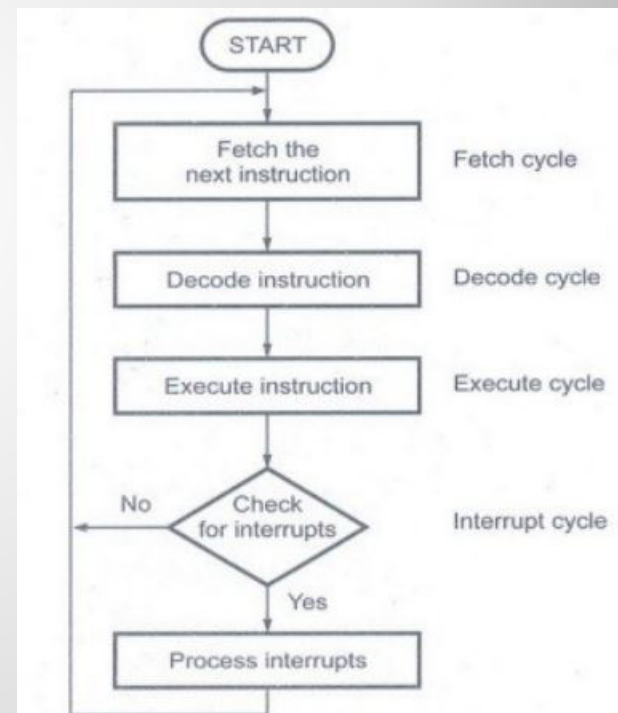
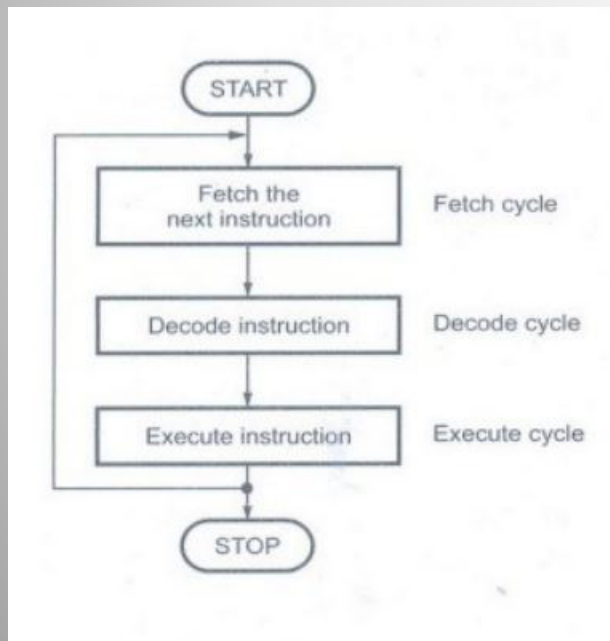


Fig. 7.1.2 Basic instruction cycle with interrupt

- **Instruction Fetch Cycle:**
 - Instruction – fetched from Memory location(address in PC)
 - placed in IR(processor)

- **Instruction Decode Cycle:**

- Opcode – decoded/examined

- **Instruction Execution Cycle:**

- Specified operation – processor
 - Fetching operands – Memory/process register,
performing Arithmetic/Logical – storing result
 - PC gets incremented

Implementation of a subset of the core MIPS instruction set

- The memory-reference instructions: **load word (lw)** and **store word (sw)**
- The arithmetic-logical instructions: **add, sub, AND, OR, etc**
- The branch instructions: **branch equal (beq)** and **jump (j)**

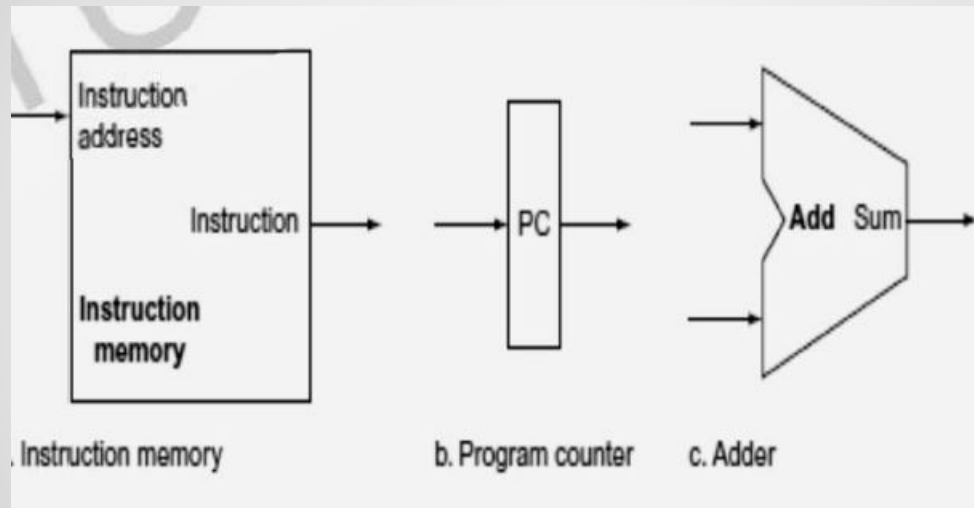
BUILDING A DATA PATH

Data Path: Collection of functional units.

A larger datapath can be made by joining more than one datapaths using MUX.

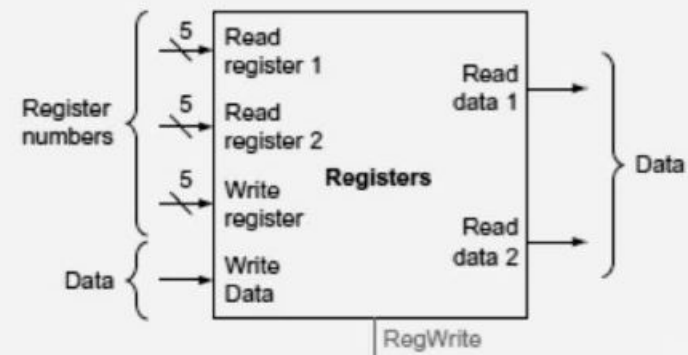
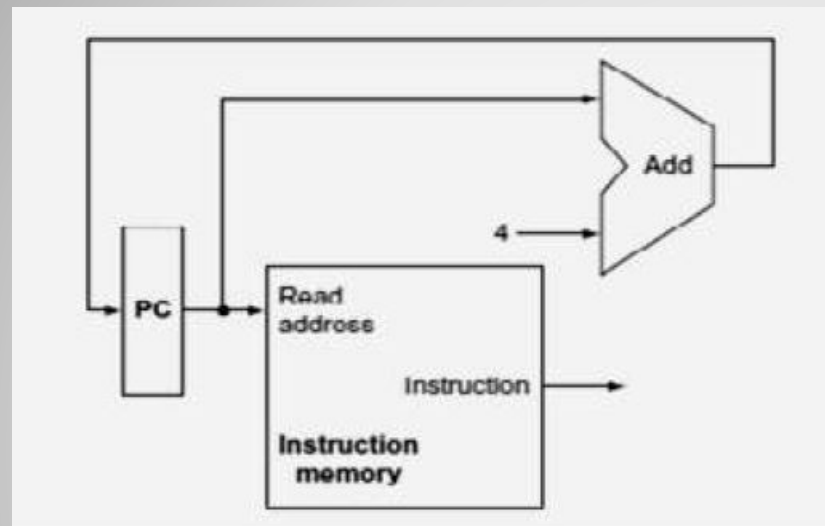
Three Instructions: R type, Load/Store,
Jump

● R Type

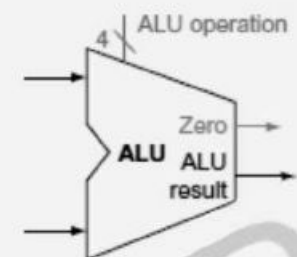


Major Components:

- **PC (Program Counter):** Holds the address of the next instruction to be executed.
- **Instruction Memory:** Fetches the instruction from memory based on the value of the PC.
- **Adders:** One adder adds 4 to the PC to point to the next instruction. The other adder is used for branch instructions.
- **Registers:** Two registers (Read register 1 and 2) to fetch operands. Write register to store the result of operations.

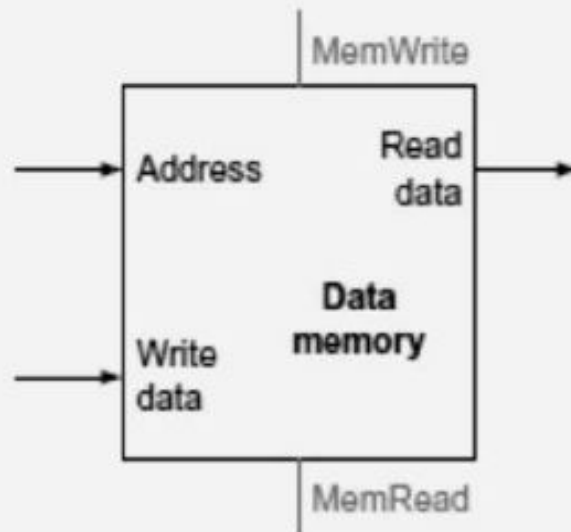


a. Registers

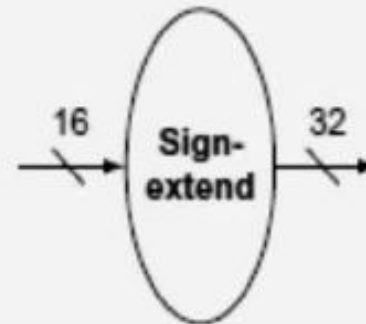


b. ALU

Load and Store Instruction



a. Data memory unit

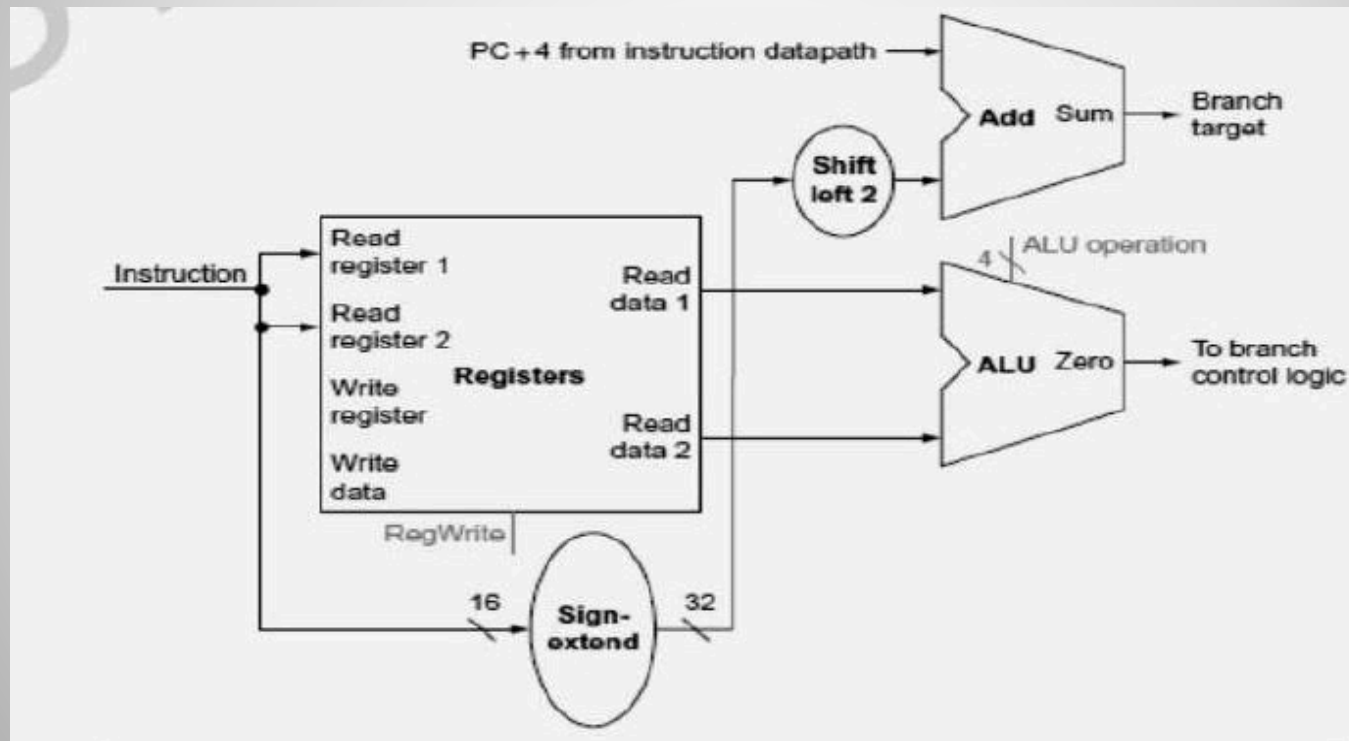


b. Sign extension unit

Load and Store Instruction

- ALU, Registers, Sign Extend, Data Memory
- **Sign-extend:** Extends 16-bit immediate values to 32 bits for ALU operations.
- **Data Memory:** Stores or retrieves data from memory during load/store instructions.

BRANCH Instruction

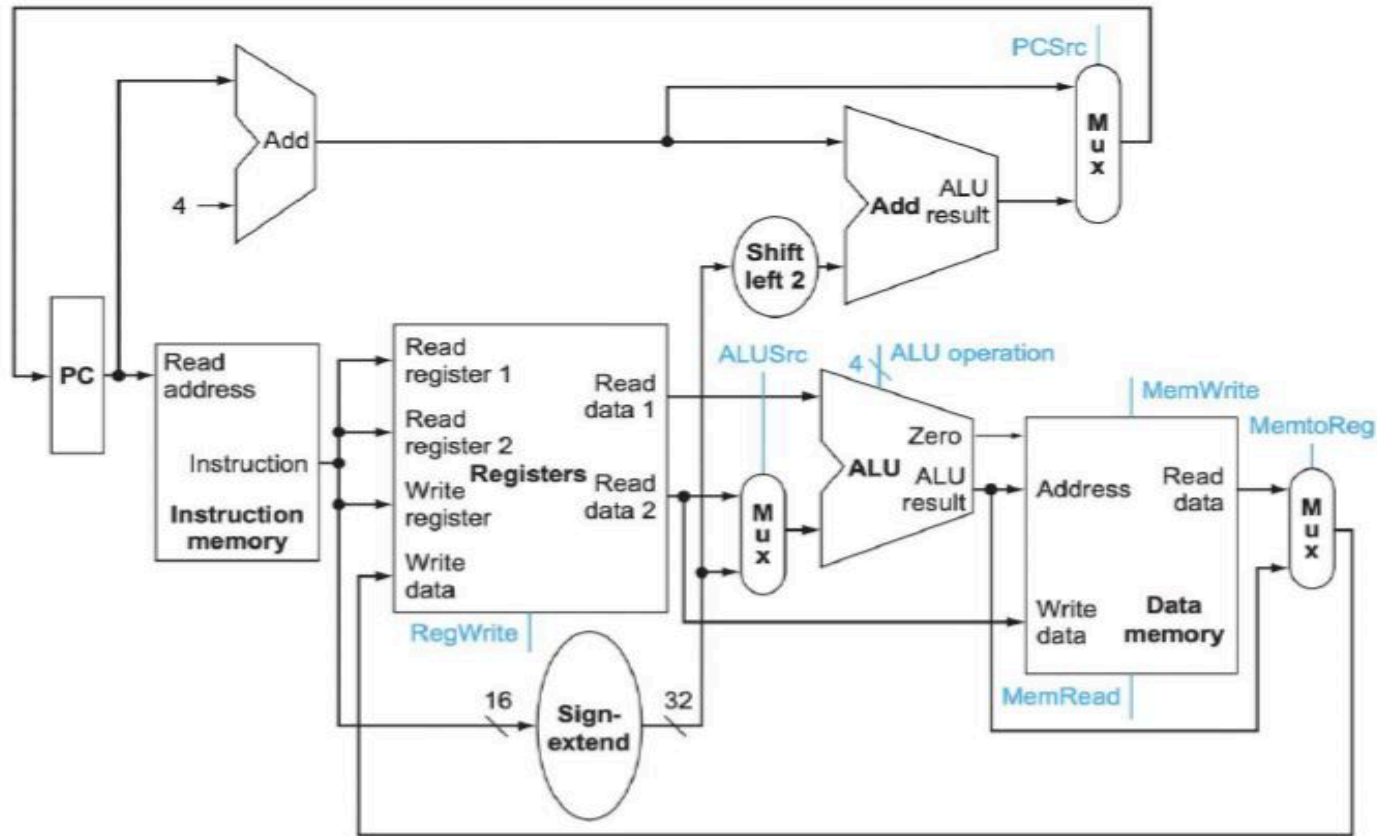


BRANCH Instruction

- Eg. **beq \$t1,\$t2,offset**
- The **beq instruction** has **three operands**, two registers that are compared for **equality**.
- **If** [contents of t1 == contents of t2]
Compute target using offset and take branch. (ALU used to check equality)
- **Else** - Proceed with next instruction

- 1. **Separate adder** - Used for computing the branch target address.
- 2. **Shift left** - Used to add two zeroes to the low-order end of the sign-extended offset field.

Overall DATAPATH



- **Multiplexer** - Select the circuit combination as per the nature of instruction.

CONTROL UNIT

- ❖ **The Control Unit was historically defined as one distinct part of the 1946 reference model of Von Neumann architecture.**
- ❖ **The function of control unit is to generate relevant timing and control signals to all operations in the computer.**
- ❖ **Control Unit is “the brain within the brain”.**
- ❖ **It controls the flow of data between the processor and memory and peripherals.**
- ❖ **The examples of devices that require a control unit are CPUs and graphics processing units (GPUs).**

The Setting of Control Signal depends on :

- Contents of the Step Counter
- Contents of the Instruction Register
- The result of a computation or a comparison operation
- External input signals, such as interrupt requests

TYPES

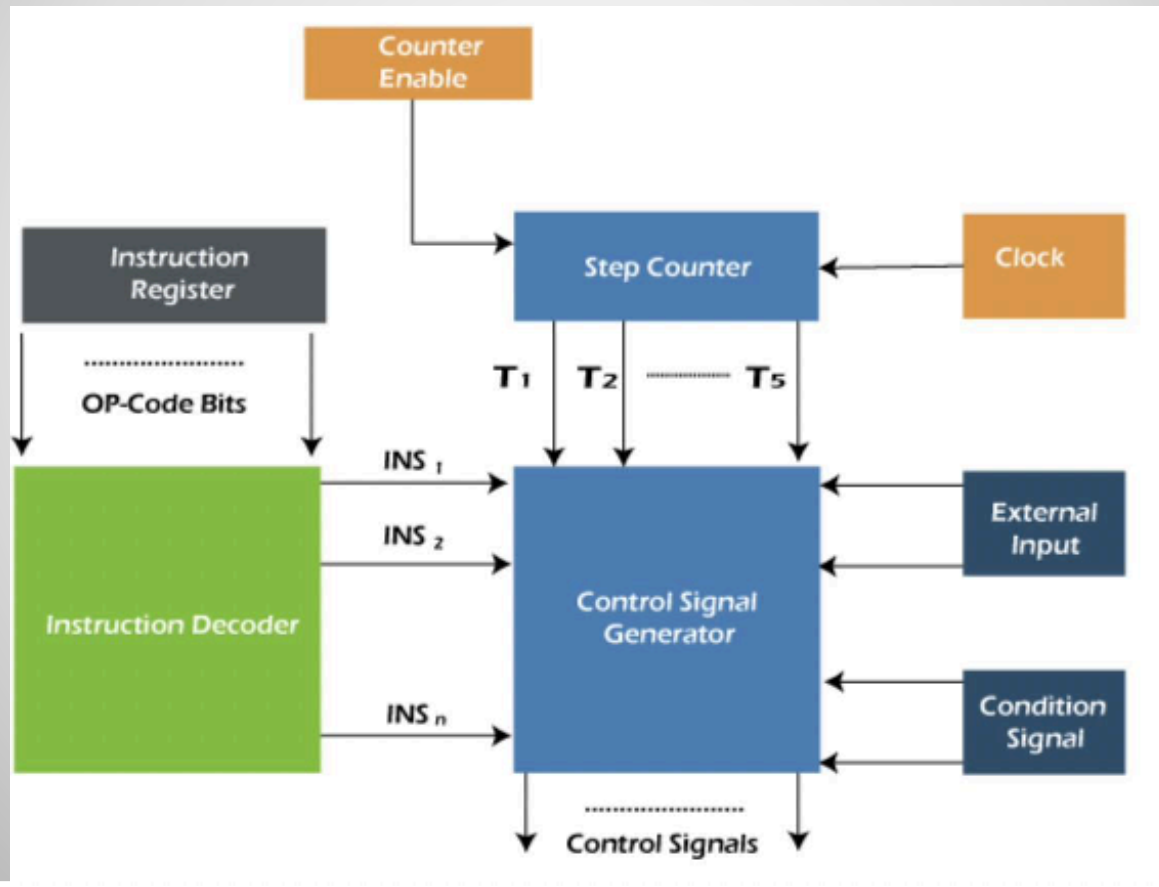
Control unit generates control signals using one of the two organizations :

- ❖ **Hardwired Control Unit.**
- ❖ **Micro-programmed Control Unit.**

Hardwired Control Unit

- ❖ **Hardwired control units are implemented through use of sequential logic units or circuits like gates , fliflops , decoders in hardware.**
- ❖ **Hardwired control units are generally faster than micro-programmed designs.**
- ❖ **The hardwired approach has become less popular and they were difficult to design.**

Hardwired Control Unit



Instruction Register:

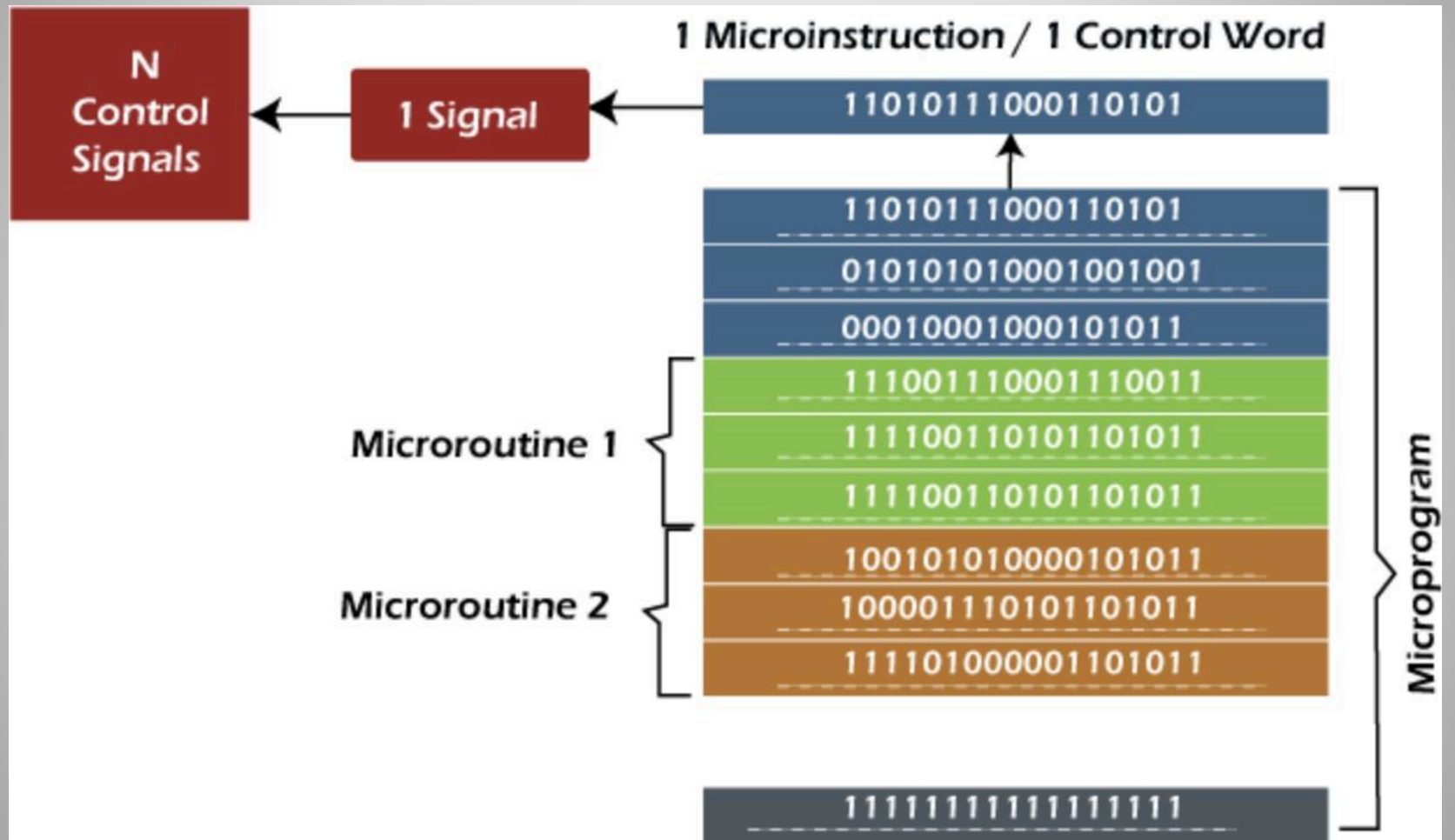
- This register holds the **current instruction** fetched from memory.
- It sends the **opcode bits** (operation code) to the **Instruction Decoder** to determine what instruction needs to be executed.

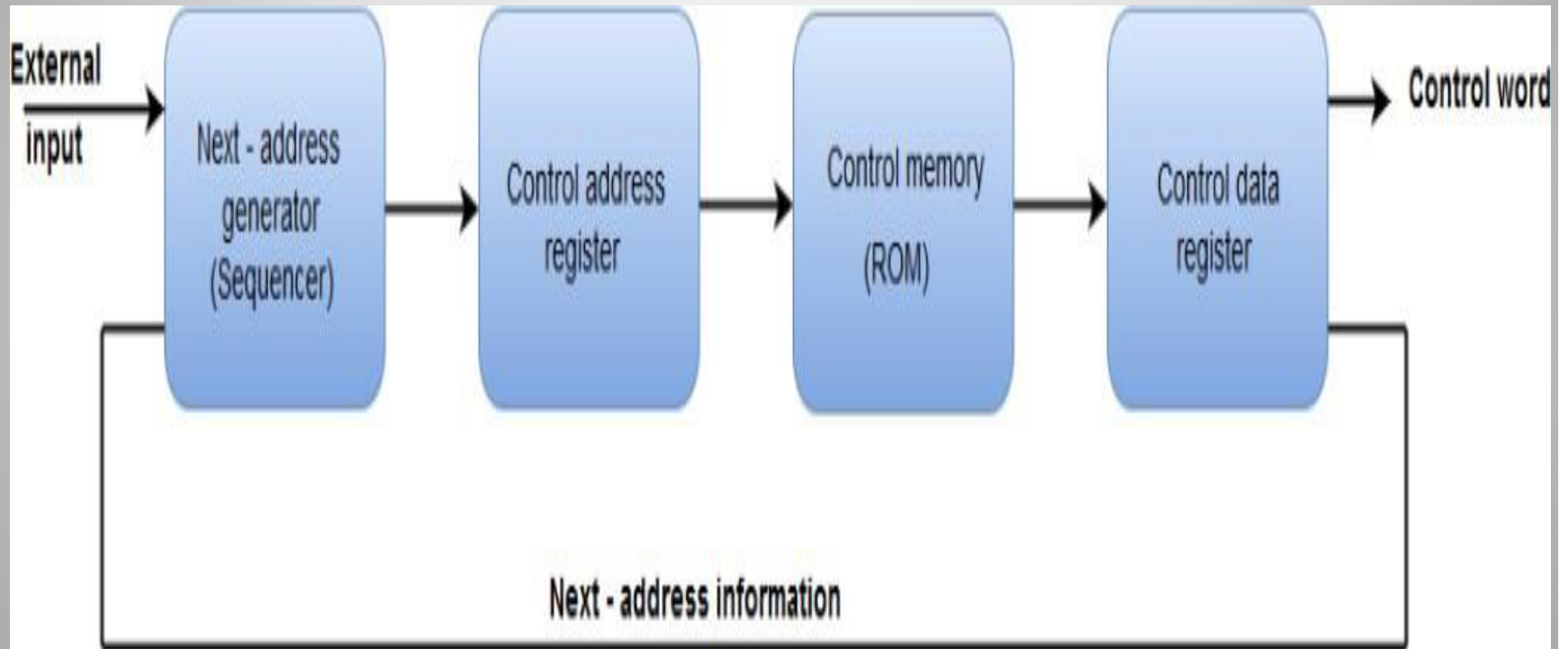
Instruction Decoder:

- This component decodes the opcode from the instruction to understand the type of operation to be performed.
- The decoded instructions are sent to the **Control Signal Generator** as **$INS_1, INS_2, \dots, INS_n$** which represent different instructions or control actions.

Step Counter:

- The step counter is controlled by a **Clock** and **Counter Enable** signals.
- It generates time steps **$T_1, T_2, \dots, T_5$** , which are essentially phases of instruction execution (like fetching, decoding, executing, etc.).





PIPELINING

- Implementation technique – multiple instructions are overlapped in execution.
- Process of arrangement of hardware elements of CPU – overall performance is 
- Pipelined processor – Simultaneous execution of more than one instruction
- Non Pipelined Execution & Pipelined Execution

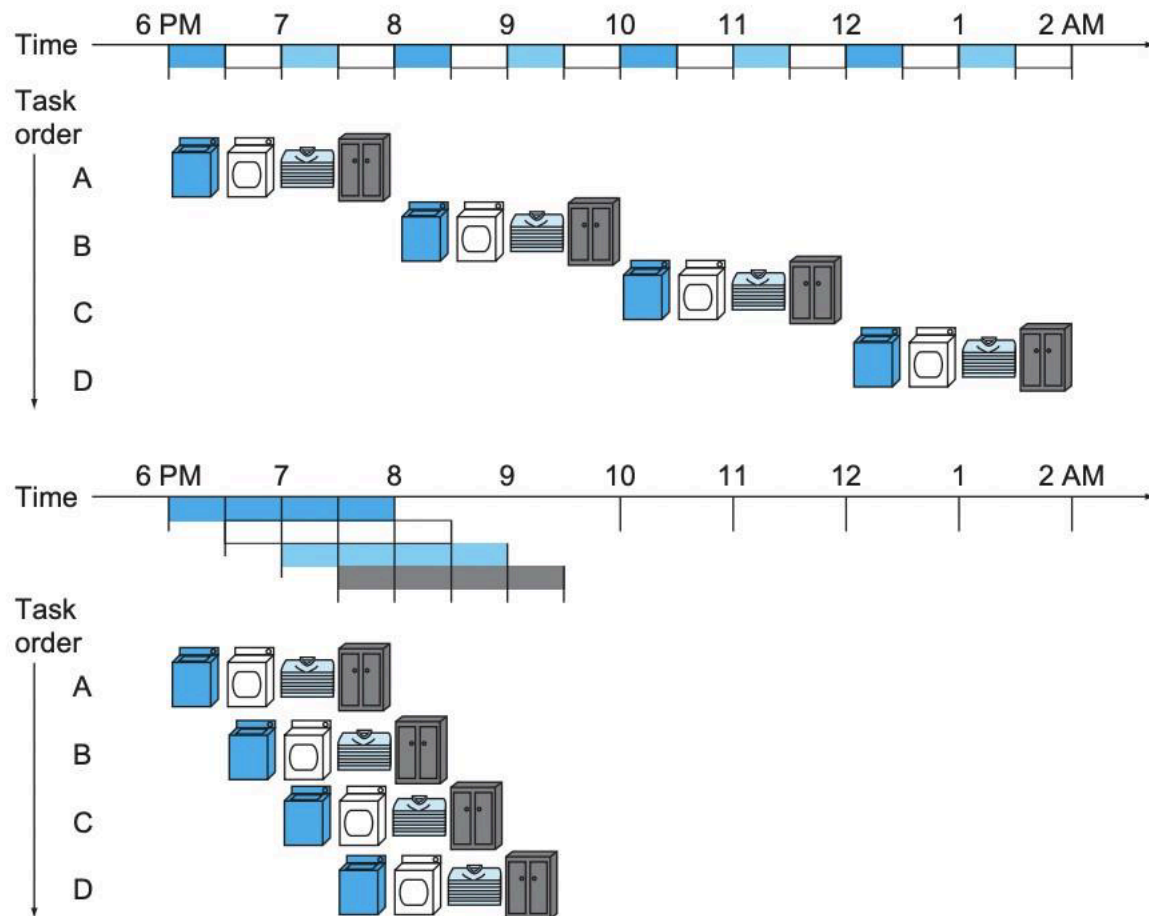
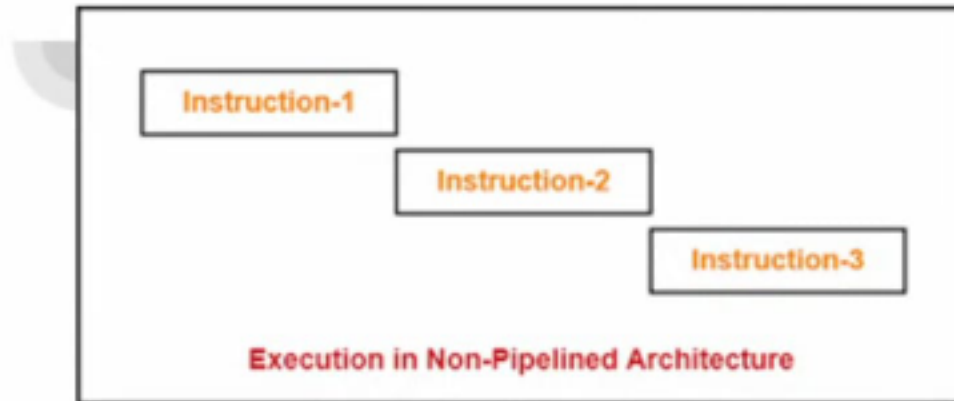


FIGURE 4.25 The laundry analogy for pipelining. Ann, Brian, Cathy, and Don each have dirty clothes to be washed, dried, folded, and put away. The washer, dryer, “folder,” and “storer” each take 30 minutes for their task. Sequential laundry takes 8 hours for 4 loads of wash, while pipelined laundry takes just 3.5 hours. We show the pipeline stage of different loads over time by showing copies of the four resources on this two-dimensional time line, but we really have just one of each resource.

Design of a Basic Pipeline

- **Pipeline** has two ends - **input end and output end**. Between these ends, there are multiple stages/segments such that the output of one stage is connected to the input of the next stage and each stage performs a specific operation.
- **Interface registers** are used to hold the intermediate output between two stages. These interface registers are also called **latch** or **buffer**.
- All the stages in the pipeline along with the interface registers are controlled by a **common clock**.

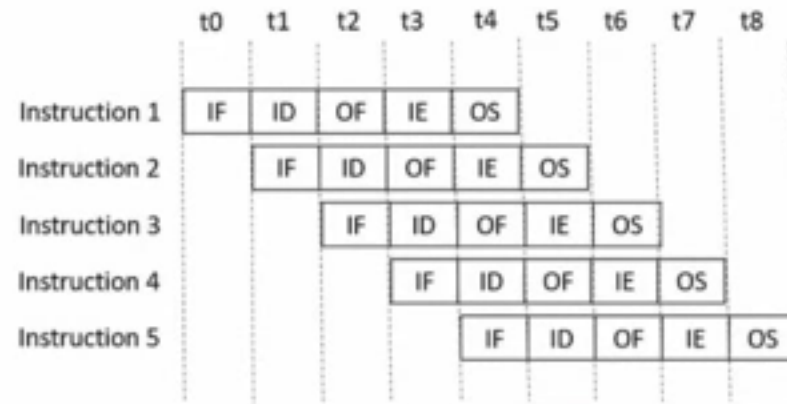
Execution in Non-Pipelined Architecture



Pipelined Representation



Pipelined Representation



Pipelining of 5 Instructions

Main points:

- Instruction pipelining is a technique that implements a form of parallelism called as instruction level parallelism within a single processor.
- Multiple instructions are executed parallelly.
- **Staging:**
 - The hardware of the CPU is split up into several functional units.
 - Each functional unit performs a dedicated task.
 - The number of functional units may vary from processor to processor.
 - These functional units are called as stages of the pipeline.
 - Control unit manages all the stages using control signals.
 - There is a register associated with each stage that holds the data.
 - There is a global clock that synchronizes the working of all the stages.
 - At the beginning of each clock cycle, each stage takes the input from its register.
 - Each stage then processes the data and feed its output to the register of the next stage.



Advantages vs Drawbacks

Advantages of Pipelining

- Instruction throughput increases.
- Increase in the number of pipeline stages increases the number of instructions executed simultaneously.
- Faster ALU can be designed when pipelining is used.
- Pipelined CPU's works at higher clock frequencies than the RAM.
- Pipelining increases the overall performance of the CPU.

Disadvantages of Pipelining

- Designing of the pipelined processor is complex.
- Instruction latency increases in pipelined processors.
- The throughput of a pipelined processor is difficult to predict.
- The longer the pipeline, worse the problem of hazard for branch instructions.

Hazards in Pipelined Architecture

There are situations in pipelining when the next instruction cannot execute in the following clock cycle. These events are called hazards, and there are three different types.

- 1) Structural Hazard
- 2) Data Hazard
- 3) Control Hazard

1. Structural Hazard

- ❖ It means that the hardware cannot support the combination of instructions that we want to execute in the same clock cycle.
- ❖ This dependency arises due to the resource conflict in the pipeline. A resource conflict is a situation when more than one instruction tries to access the same resource in the same cycle. A resource can be a register, memory, or ALU.

Instruction / Cycle	1	2	3	4	5
I ₁	IF(Mem)	ID	EX	Mem	
I ₂		IF(Mem)	ID	EX	
I ₃			IF(Mem)	ID	EX
I ₄				IF(Mem)	ID

Solution for Structural Hazard

Cycle	1	2	3	4	5	6	7	8
I ₁	IF(Mem)	ID	EX	Mem	WB			
I ₂		IF(Mem)	ID	EX	Mem	WB		
I ₃			IF(Mem)	ID	EX	Mem	WB	
I ₄				-	-	-	IF(Mem)	

Stall:

To avoid this problem, we have to keep the instruction on wait until the required resource (memory in our case) becomes available.

2. Data Hazard

Data hazards occur when instructions that exhibit data dependence, modify data in different stages of a pipeline.

I1 : ADD R1, R2, R3

I2 : SUB R4, R1, R2

Instruction / Cycle	1	2	3	4
I ₁	IF	ID	EX	DM
I ₂		IF	ID(Old value)	EX

Solution for Data Hazard - Operand Forwarding

In operand forwarding, we use the interface registers present between the stages to hold intermediate output so that dependent instruction can access new value from the interface register directly.

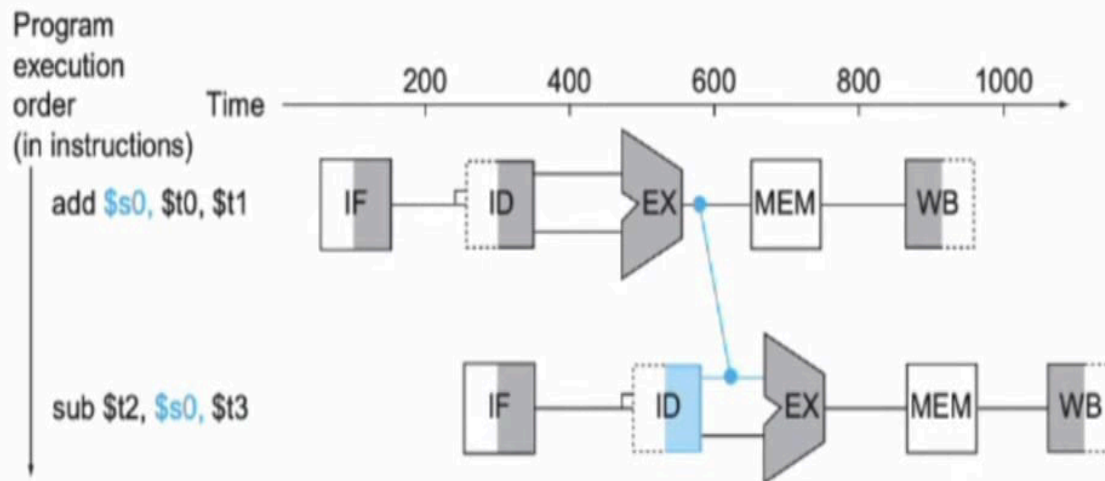


FIGURE 4.29 Graphical representation of forwarding. The connection shows the forwarding path from the output of the EX stage of `add` to the input of the EX stage for `sub`, replacing the value from register `$s0` read in the second stage of `sub`.

3.Control Hazard



- ❖ This type of dependency occurs during the transfer of control instructions such as BRANCH, CALL, JMP, etc.
- ❖ On many instruction architectures, the processor will not know the target address of these instructions when it needs to insert the new instruction into the pipeline.
- ❖ Due to this, unwanted instructions are fed to the pipeline.
- ❖ Eg. 100: I₁

101: I₂ (JMP 250)

(Jump address known after ID stage only)

102: I₃

250: B1₁

Expected output: I₁ -> I₂ -> B1₁

Instruction/ Cycle	1	2	3	4	5	6
I ₁	IF	ID	EX	MEM	WB	
I ₂		IF	ID (PC:250)	EX	Mem	WB
I ₃			IF	ID	EX	Mem
B1 ₁				IF	ID	EX



Solution for Control Hazard

1. Stall

To correct the above problem we need to stop the Instruction fetch until we get target address of branch instruction.

Instruction/ Cycle	1	2	3	4	5	6
I ₁	IF	ID	EX	MEM	WB	
I ₂		IF	ID (PC:250)	EX	Mem	WB
Delay	-	-	-	-	-	-
BI ₁				IF	ID	EX

Output Sequence: I₁ -> I₂ -> Delay (Stall) -> BI₁

2. Branch Prediction

- Predict always that branches will be untaken. When you're right, the pipeline proceeds at full speed. Only when branches are taken does the pipeline stall.
- Some branches predicted as taken and some as untaken. Eg. Loops
- Dynamic hardware predictors, in stark contrast, make their guesses depending on the behavior of each branch and may change predictions for a branch over the life of a program